UNIVERSITY
OF OSLO

Master's thesis

Rethinking Multilevel K-Way Graph Partitioning with a Partition-Aware Volume Function

Riccardo Ieva

Informatics: Programming and Systems Architecture

Department of Informatics
Faculty of Mathematics and Natural Sciences

Spring 2026

Riccardo Ieva

Rethinking Multilevel K-Way Graph Partitioning with a Partition-Aware Volume Function

Supervisors:
Xing Cai
James D. Trotter

Abstract

Here come 3–6 sentences describing your thesis.

Preface

Here comes your preface, including acknowledgments and thanks.

Contents

1 Introduction . . . 1
  1.1 Thesis Structure . . . 2
2 Motivation and Background. . . . 5
  2.1 Graph Partitioning Problem . . . 5
    2.1.1 Objective Functions . . . 5
    2.1.2 Shortcomings of the Objective Functions . . . 6
  2.2 Graph Partitioning Algorithms . . . 7
    2.2.1 Spectral Bisection . . . 7
    2.2.2 Graph Growing . . . 7
    2.2.3 Streaming Graph Partitioning . . . 8
    2.2.4 Multilevel Graph Partitioning . . . 8
  2.3 Applications of Graph Partitioning . . . 8
    2.3.1 Parallel Processing . . . 8
    2.3.2 Complex Networks . . . 8
    2.3.3 VLSI Physical Design . . . 9
    2.3.4 Neural Networks . . . 9
3 Multilevel K-Way Graph Partitioning . . . 11
  3.1 Why Multilevel K-way? . . . 11
4 Communication Volume Calculation . . . 13
  4.1 Basic definitions . . . 13
  4.2 METIS gain calculation . . . 14
  4.3 Partition specific gain calculation . . . 17
  4.4 Examples . . . 18
    4.4.1 Two Partition example . . . 18
    4.4.2 Three partition example . . . 19
    4.4.3 Case 4 example . . . 21
  4.5 Implementation . . . 23
5 Results . . . 25
6 Conclusions and Future Work . . . 27
7 Results . . . 29
8 Future Work . . . 31

List of Figures

2.1 Vertex A has a communication volume of 1 but an edge cut of 2. Each color represents a partition. 6

4.1 Vertex A has a communication volume of 2. Each color represents a partition. 14
4.2 Vertex A has a communication volume of 1. Each color represents a partition. 14
4.3 Graph with a total communication volume of 8. 14
4.4 Graph with a volume of 4, each partition has a volume of 2. 18
4.5 Graph with a volume of 4, each partition has a volume of 2 after the move. . 18
4.6 Graph with a total communication volume of 8. 19
4.7 Graph has a volume of 6 after *B* is moved to the blue partition 21
4.8 Graph has a volume of 8 after *B* is moved to the green partition 21
4.9 Graph with a total communication volume of 8. 21
4.10 Graph has a volume of 5 after the move. 23
4.11 Graph has a volume of 6 after the move. 23

List of Tables

4.1 $k = 32$ 24

List of Algorithms

1 METIS gain calculation 16

Acronyms

GP Graph Partitioning
GPP Graph Partitioning Problem
BFS Breadth-First Search
SGP Streaming Graph Partitioning
CPU Central Processing Unit
MPI Message Passing Interface

Chapter 1

Introduction

The demand for computational resources in modern scientific and engineering applications keeps increasing, making parallel computing an essential area of development. While processor performance continues to improve, the growth in single-core performance has slowed considerably since the end of Dennard scaling [19], making it impractical to tackle the largest problems on a single processor alone. Instead, it has become necessary to distribute these workloads across the many processors of a parallel computing system with the help of graph partitioning algorithms.

In order to compute large communicating tasks in parallel computing, many models have been proposed that represent them as graphs or hypergraphs [5, 6, 8, 17, 40]. The vertices correspond to units of computation and the edges encode data dependencies between them [17]. Efficiently distributing the computation then becomes a question of how to partition this graph among processors: balancing the workload evenly while minimizing the communication between them. This is also known as the *graph partitioning problem*. Although graph partitioning is known to be NP-complete, a variety of algorithms have been proposed that produce good quality solutions in practice by using heuristics [18, 22, 24, 25]. Beyond parallel computing, graph partitioning plays a critical role in numerous other application areas, including task scheduling, neural network simulations, operations research and more.

In 2005, Herb Sutter published *Software and the Concurrency Revolution* [38], arguing that the future of software development lies in concurrency. As computational demands continue to grow, our programs and requirements consistently outpace hardware improvements. Sutter highlights the need for better parallel programming tools, given the inherent difficulty of developing parallel solutions. A decade later, on July 29, 2015, President Obama issued Executive Order 13702, establishing the National Strategic Computing Initiative (NSCI) to ensure continued United States leadership in high-performance computing [11], underscoring the importance of this field and the need for sustained research. Graph partitioning algorithms are central to making this possible.

Most modern partitioning algorithms minimize for *edge cut* (see Section 2.1.1), seeking to minimize the number of inter-partition edges. The appeal of this metric lies in its simplicity: it is easy to compute, and for the structured meshes that dominate scientific computing, it correlates reasonably well with actual communication costs [17]. However, edge cut has several well known shortcomings as a representation for communication [16, 17]. First, it over counts *communication volume* (see Section 2.1.1): a vertex with multiple edges crossing into the same partition needs to be communicated only once, yet each edge is counted separately when using edge cut. Second, the metric ignores data communication latency, the startup cost when sharing data between

partitions, which can dominate communication cost when the size of the communicated data is small. Third, and most relevant to this work, minimizing the *total* edge cut does not account for how communication is distributed across partitions. In a parallel application, performance is generally limited by the slowest processor [17], meaning that even if balanced globally, some processors might bear a greater load than others, creating bottlenecks.

Karypis and Moulitsas highlighted the need for partitioning algorithms that account for heterogeneous architectures and proposed using *communication volume* as an optimization objective instead of edge cut [30]. This communication volume objective was subsequently added into the METIS software package. However, like the edge cut objective, METIS focuses on the global communication volume without considering how volume is distributed among partitions. Regardless of the architecture, focusing only on the total volume of a graph may lead to bottlenecks in some partitions.

Efficiently distributing computation across processors is central to high-performance computing, and graph partitioning is the standard technique for achieving this. In *Graph Partitioning and Parallel Solvers: Has the Emperor No Clothes?* [16], Hendrickson concludes that one of the key needs is to develop a more accurate metric for communication cost. Researchers have proposed alternative objectives that target per-processor communication metrics rather than global totals. An example is UMPa [8], which minimizes the maximum data sent by any single processor in a distributed memory setting.

As Hendrickson noted, objectives of this kind are harder to optimize than the standard edge cut, which partly explains their limited adoption. Despite these difficulties, in this thesis, we present a new objective called *NVOL*. NVOL minimizes the *per-partition* maximum volume, aiming to reduce uneven communication loads. In this thesis, we will answer two questions: Can a partition-aware objective produce better balanced communication loads than a global one? Can the performance of a parallel application be improved in practice by using a partition-aware objective?

1.1 Thesis Structure

Chapter 2: *Motivation and Background*

> Chapter 2 covers some of the fundamental topics in graph partitioning. It also shows some of the algorithms used today and some applications of graph partitioning algorithms.

Chapter 3: *Multilevel K-Way Graph Partitioning*

> Chapter 3 discusses the Multilevel K-way graph partitioning algorithm used to develop NVOL. This approach achieves a good balance between quality and run time, making it a popular option for large graphs.

Chapter 4:

> ...

Chapter 5:

> ...

Chapter 6:

...

Chapter 7:

...

Chapter 2

Motivation and Background

This chapter gives a description of what the *graph partitioning problem* (GPP) is, as well as what objectives are used to solve this. It also discusses some of the common practices in the field with some examples of concrete application areas for *graph partitioning* (GP).

2.1 Graph Partitioning Problem

The GPP is to partition a graph into k subsets of vertices subject to balance requirements. Formally, given an undirected graph $G = (V, E)$ with $|V| = n$ vertices and $|E| = m$ edges. The goal is to partition V into k disjoint subsets $V_1, V_2, \ldots, V_k$ such that $V_i \cap V_j = \emptyset$ for $i \neq j$ [24]. A balance constraint requires that all subsets have approximately equal weight. Using an imbalance parameter ϵ, this constraint requires that each subset satisfies

$$|V_i| \leq \left\lceil \frac{n}{k} \right\rceil \cdot (1 + \epsilon), \quad i = 1, 2, \ldots, k.$$

The parameter ϵ represents the fractional excess weight a subset may have beyond the perfectly balanced size $\lceil n/k \rceil$. When $\epsilon = 0$, the partition must be perfectly balanced [4]. The goal of GPP is to minimize a given objective function while satisfying the balance constraint. GP is proven to be NP-complete, meaning that for large graphs an approximate solution can only be achieved through heuristics or approximation algorithms [14]. In GP, the most common practice is to use heuristics. Approximation algorithms, regardless of their high theoretical importance, either under perform compared to state-of-the-art GP solvers, or are too slow for large graphs [4].

Another common representation of computational tasks in GP is *hypergraphs*. A hypergraph is a generalization of a graph where an edge, also called a hyperedge, can connect any number of vertices [3]. Similarly, GP divides the vertices into partitions while trying to maintain roughly equal size for each. However, given this more complex structure, utilizing hypergraphs requires more complex algorithms both in terms of implementation and running time. For the scope of this thesis, we will focus on standard GP; however, most of the techniques presented in this section (See 2.2) ¨have been or can be transferred to a hypergraph solution.

2.1.1 Objective Functions

The most common objective in GPP is to minimize the total edge cut [4]. An edge $\{u, v\} \in E$ is said to be *cut* if its endpoints belong to different partitions, i.e., $u \in V_i$

and $v \in V_j$ with $i \neq j$. The total edge cut is then the sum of all such cut edges across partitions. This also applies for graphs with edge weights, where the total edge cut is then the sum of the edge weights of cut edges.

Another common objective is communication volume, which provides a more accurate measure of the data exchange between partitions of a communicating task. For a partition V_i, the outgoing communication volume is defined as $\text{comm}(V_i) = \sum_{v \in V_i} D(v)$, where $D(v)$ denotes the number of distinct partitions, other than the one containing v, in which v has at least one edge to another vertex, also known as a neighbor.

Despite being a less precise measure for actual communication cost, edge cut has been widely adopted as the standard objective. Buluç et al. [4] attribute this to two factors: cut optimization appears to be easier in practice, and for graphs with high structural locality, the cut tends to perform similarly to other objectives. Additionally, alternative objectives complicate the use of multilevel approaches making them harder to implement.

2.1.2 Shortcomings of the Objective Functions

Even though edge cut is accepted as the standard, it comes with several shortcomings. Most notably, it does not distinguish between multiple edges connecting to the same neighboring partition. Consider Figure 2.1, where each color represents a partition. Vertex A has two edges, both leading to the blue partition. Calculating the edge cut, A contributes 2 to the total edge cut. The communication volume, however, is only 1, since both edges connect to the same partition. In general, edge cut can overestimate the actual communication burden, particularly for vertices with many neighbors in a single remote partition.

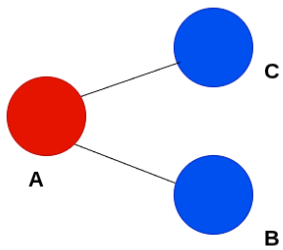

Figure 2.1: Vertex A has a communication volume of 1 but an edge cut of 2. Each color represents a partition.

The performance of a parallel application is generally limited by its slowest processor. This means that even if the computational work is balanced, the communication effort may not be [4]. In such cases, it is more meaningful to minimize the maximum communication load incurred on a single processor rather than the total load across all processors. The standard edge cut metric nor the total communication volume captures this type of objective.

Furthermore, on many architectures the time to send data depends not on geographic distance but on the number of network switches the data, commonly called a message in this setting, must traverse. Although modern machines employ cut-through or wormhole routing to move individual messages quickly between distant processors, the communication network handles many messages simultaneously, and a message between distant processors occupies links that are unavailable to other traffic. Minimizing edge

cut does not account for this locality of communication, and a partition that places frequently communicating processors far apart in the network topology can lead to significant message contention [4, 17].

Finally, the time to send a message on a parallel computer depends on both the latency (startup cost) and the message size. GP approaches typically aim to minimize the total communication volume but not the total number of messages sent. Depending on the machine architecture and problem size, message latencies can dominate message volume, making the number of distinct messages also relevant cost factor [4, 17]. A few big messages might be more efficient than many small ones.

Regardless of the objective function chosen, GP remains a fundamental tool across a wide range of domains. The following sections discuss some of the most used GP algorithms and some of their application areas.

2.2 Graph Partitioning Algorithms

In this section we will present some of the most popular GP algorithms used today. The goal is to show some of the differences in the approaches and to highlight what their usage is. These are however only *some* of the existing GP algorithms, a more complete list can be found in [4, 7].

2.2.1 Spectral Bisection

Spectral Bisection was one of the first methods used to split a graph into two partitions and is still used today [4]. Its approach is to use the eigenvector, also known as *Fiedler vector*, of the second smallest eigenvalue of the Laplacian matrix to deduce global information of the connectivity of the graph [12]. In order to form the two partitions, the median value $\overline{m}$ of the Fiedler vector is calculated, and every vertex with a value smaller or equal to $\overline{m}$ is assigned to one partition, and the rest to the other one.

The main computational cost of the spectral method lies in the calculation of the Fiedler vector. To optimize this, Barnard and Simon [1] have used a multilevel method to produce an approximation of the Fiedler vector. The spectral method has also been extended to produce a partition for more than only two partitions and to support edge and vertex weights [18].

2.2.2 Graph Growing

Another bisection approach is called graph growing [23]. Graph growing is mainly based on breadth-first search (BFS). The core idea is to start from a random vertex and to use BFS to construct a partition. BFS is run until half the vertices are assigned to one partition, the remaining vertices are then assigned to the other one. The choice of starting vertex significantly affects the result, meaning that this algorithm tends to require multiple restarts to obtain a good solution. The graph growing approach has also been extended to produce more than two partitions, called *Bubble framework* [10]. In this version k starting vertices are chosen, evenly distributed in the graph. Starting from these vertices, k BFS traversals are performed. Throughout, *Bubble* uses local search algorithms to balance the partitions in the graph. At the end of an iteration, a new starting vertex is chosen for all partitions. The new starting vertex is the vertex with the minimum total distance to all other vertices in the same partition. The algorithm is repeated until no new changes occur in the choice of starting vertices,

or no improvement for the partitions is found. The major drawback of this approach is its high computational cost.

2.2.3 Streaming Graph Partitioning

Streaming data models, where input is processed on the fly rather than all at once, have gained popularity due to their small memory requirements. In the field of GP, the input graph is also passed as a stream, requiring much less space than the graph size [4]. Streaming graph partitioning (SGP) is also very fast, even faster than multilevel algorithms (See 2.2.4). However, these algorithms produce lower quality partitions compared to multilevel approaches. They are nonetheless useful in applications where speed is important, and an initial partition can be obtained from a stronger GP algorithm. For more details about SGP, we refer the reader to [33, 37, 39].

2.2.4 Multilevel Graph Partitioning

Multilevel GP is considered to be the most successful heuristic for partitioning large graphs [4, 23, 25]. Because of this, multilevel GP was chosen as the base on which to build our new objective function NVOL. In short, the idea is that a graph is coarsened to a smaller graph. Once the graph is at its smallest, an initial partition is performed, and then it is iteratively uncoarsened to the original size. During the uncoarsening, heuristics are applied to improve the partitions. The intuition behind multilevel GP is that it is easy to run intensive partitioning algorithms on the small graph without affecting the total runtime. Additionally, starting from an already good solution, the refinement stage is not costly to perform and can quickly improve the partitions. Chapter 3 will explain in detail, providing the necessary background for understanding how NVOL is integrated into the framework.

2.3 Applications of Graph Partitioning

Graph partitioning arises in a broad variety of domains. This section highlights several of the most prominent application areas, though the list is by no means exhaustive.

2.3.1 Parallel Processing

Perhaps the most widespread application of GP is distributing computational work across the processors of a parallel machine [27]. GP is the standard approach in scientific computing applications such as sparse direct and iterative solvers. When the problem domain remains fixed throughout the computation, the partitioning can be performed once at the outset; this is known as static partitioning [4]. Other common applications in parallel processing include eigenvalue computations [2], breadth-first search [3], and algorithms for PageRank and connected components [35].

2.3.2 Complex Networks

GP is also relatively popular in the field of complex networks [4]. Complex networks, in this context, refer to weighted graphs with nontrivial structure arising from real world or modelling processes [32]. The objectives applied in this field are usually domain specific, making it hard to develop optimization algorithms.

Several application domains illustrate this breadth. In power grids, disturbances and cascading failures may lead to blackouts. GP can be used to isolate self-sufficient islands to prevent propagation of an outage [28]. Cut based objectives are sometimes extended to enhance the robustness of the system [29].

Geographically embedded networks have experienced a growth in requirements with the advances in location-aware devices (such as GPS). It is now vital that algorithms be able to analyze networks like these extremely fast. In these networks the vertices commonly represent entities in the real world tied to geographic places. The cut-based approach therefore needs to be reinforced with spatial contiguity constraints [4, 29].

Many biological systems can be modeled as graphs [4], protein-protein interaction or gene co-expression graphs, for example. Partitioning serves multiple purposes ranging from data reduction to the detection of biological processes. More about this field can be found in [20].

In social networks, community detection is one of the most popular topics. In contrast to traditional GPP, the number of subgroups or partitions is not known a priori [4]. Apart from this, GP has contributed greatly to the development of community detection algorithms [13, 31].

2.3.3 VLSI Physical Design

VLSI circuit design has historically been one of the primary application areas for graph partitioning [4]. The objective is to decompose a circuit into smaller subcircuits, ranging from gate arrays to complete integrated circuits, while minimizing the total interconnection cost, typically measured as the weight of connections crossing partition boundaries. In this formulation, nodes represent cells (i.e., basic logic or functional units such as gates) and edges represent wires. Scalability is essential, as circuits can contain millions of modules and partitioning constitutes a bottleneck in the overall design flow. Beyond the basic cut objective, practical VLSI partitioning must also respect domain specific constraints such as I/O placement, groups of cells that are required to remain in the same block, and upper bounds on the cut size between block pairs. Further details can be found in [9, 21].

2.3.4 Neural Networks

Graph neural networks (GNNs) extend deep learning to graph structured data, enabling the processing of irregular domains that traditional neural network architectures cannot readily handle. However, as graph sizes grow, training GNNs becomes increasingly expensive due to rising communication overhead, particularly in distributed or multi-device settings. Graph partitioning offers a natural approach to this scalability challenge by distributing subgraphs across devices while minimizing inter-device communication.

A recent example is CaPGNN [36], a parallel full batch GNN training framework designed for multi-GPU servers. CaPGNN combines resource aware graph partitioning with an adaptive caching mechanism that exploits both CPU and GPU memory to reduce redundant data transfers. By taking into account the heterogeneous computational and communication capacities of individual GPUs, the framework achieves substantial reductions in training time with minimal impact on model accuracy. While this is only one instance of a broader trend, it illustrates how classical graph partitioning techniques are finding renewed relevance in the context of modern machine learning workloads.

Another recent example is ParDNN [34], an automatic, generic, and non-intrusive partitioning strategy for Deep Neural Networks represented as graphs. Like CaPGNN,

this strategy has the goal of minimizing the training time of the model without affecting accuracy. Through their experiments, Qararyah et al. show significant improvements compared to related work in efficiency, superlinear scaling of batch size and training throughput.

Chapter 3

Multilevel K-Way Graph Partitioning

This chapter presents the Multilevek K-way Graph Partitioning algorithm which is one of the most widespread GP algorithms. Throughout each section METIS own implementation will be mentioned both for the edge cut and communication volume metric.

3.1 Why Multilevel K-way?

There are many algorithms that are able to produce good partitions.The techniques vary from global methods that partition the graph directly, to breadth first search to methods from computational geometry and linear algebra [4]. The main idea behind multilevel k-way partitioning is to coarsen down the graph to a smaller graph first. On this smaller graph an initial partition is performed on the small graph before it is uncoarsened back up to its original size. For each uncoarsening level the partition is refined. The next sections in this chapter will go in detail on how METIS implements this approach.

Most tools and algorithms implement the *multilevel* heuristic [15]. Its popularity can be attributed to the fact that this approach tends to perform better than others for large graphs [4, 23, 26].

This section will present the multilevel approach as the most important in parallel computing
Coarsening phase
Initial partition
Uncoarsening and refinement
All the way with examples of how Metis is implemented, especially for the volume objective.

Chapter 4

Communication Volume Calculation

To support the new optimization objective for NVOL, it became necessary to introduce a new procedure for calculating communication volume. The existing implementation in METIS focuses on optimizing the total communication volume between partitions. Therefore it only evaluates the net volume after a potential move, instead of how each partitions volume changes by the move. In contrast, the optimization approach introduced in this work delivers detailed information about how individual partitions are affected by a potential move.

4.1 Basic definitions

To facilitate understanding of the implementation (see Section 4.5), we begin by introducing a formal definition of communication volume. Let $G = (V, E)$ be a graph, and let $P[v]$ be the partition of vertex v for any vertex $v \in V$. A vertex u is defined as a neighbor of v if there exists an edge $\{u, v\} \in E$. Any vertex adjacent to at least one vertex belonging to a different partition is said to lie on the boundary. We denote the boundary set by $V_b \subseteq V$. Formally, a vertex $v \in V_b$ if there exists at least one neighbor $u \in V$ such that $\{v, u\} \in E$ and $P[v] \neq P[u]$.

For each boundary vertex $v \in V_b$, let V_v^{nbr} denote the set of distinct partitions (excluding $P[v]$) among its neighboring vertices. The number of partitions contained in V_v^{nbr}, defined as $|V_v^{\text{nbr}}|$, is the outgoing communication volume of vertex v. Note that the number of edges incident to v is not directly relevant for this measure, only the number of distinct neighboring partitions. In Figure 4.1, vertex A has a communication volume of 2, as it is connected to vertices in two distinct partitions. In contrast, Figure 4.2 illustrates a case where vertex A has two edges, both connecting to vertices within the same partition. Consequently, the communication volume of vertex A is 1, since only a single distinct external partition is involved.

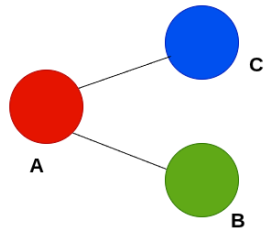

Figure 4.1: Vertex A has a communication volume of 2. Each color represents a partition.

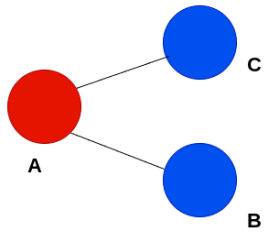

Figure 4.2: Vertex A has a communication volume of 1. Each color represents a partition.

The total communication volume of the graph is obtained by summing the outgoing volumes over all boundary vertices:

$$\text{total_volume} = \sum_{v \in V_b} \left| V_v^{\text{nbr}} \right| \text{vsize(v)} \tag{4.1}$$

Here, vsize(v) denotes the weight associated with vertex v, which scales its contribution to the outgoing communication volume. In Figure 4.3, all vertices are assumed to have a unit weight. The boundary set is defined as $V_b = \{A, B, C, D, E\}$. Note that vertex F is not part of this set, as it has no edges connecting to vertices in other partitions than its own.

Applying Equation 4.1, the total communication volume is computed by summing the contributions of all vertices in the boundary set. Vertices A, C, and D each contribute a volume of 2, while vertices B and E each contribute a volume of 1. This results in a total communication volume of 8.

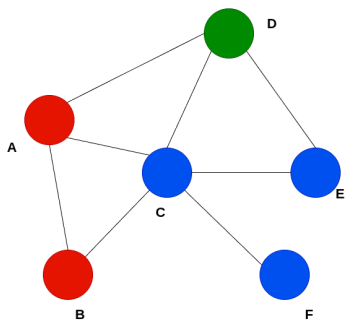

Figure 4.3: Graph with a total communication volume of 8.

4.2 METIS gain calculation

To determine which vertices should be moved, and to which partitions, METIS evaluates the gain associated with each potential move. The gain represents the change in total communication volume resulting from relocating a vertex to a new partition. A positive gain indicates a reduction in total communication volume, whereas a negative gain corresponds to an increase.

Since communication only arises when an edge connects two vertices belonging to different partitions, a vertex move can only affect its neighboring partitions. For each boundary vertex $i \in V_b$, METIS examines all adjacent vertices $v \in V_i^{\text{adj}}$, where $V_i^{\text{adj}} \subseteq V$

denotes the set of vertices adjacent to i. When considering moving a boundary vertex i, the gain calculation depends on the relationship that i and each of its neighboring vertices have with other partitions. The effect on volume depends on which partitions they share. In particular, we define for every vertex i and every neighbor vertex v

$$V_{iv}^{i} = V_i^{\text{nbr}} \setminus (V_v^{\text{nbr}} \cup P[v]) \quad \text{and} \quad V_{iv}^{\text{nbr}} = (V_i^{\text{nbr}} \cap V_v^{\text{nbr}}) \cup P[v] \tag{4.2}$$

representing partitions exclusive to i and partitions shared by i and v, respectively. In both cases, we add in $P[v]$ as this partition is shared even though it is not part of the V_v^{nbr} set. A vertex v has access to its own partition $P[v]$, meaning that $P[v]$ needs to be removed in V_{iv}^{i} and added in V_{iv}^{nbr} to precisely determine shared partitions. Notice how both sets are subsets of $V_i^{\text{nbr}} = V_{iv}^{i} \cup V_{iv}^{\text{nbr}}$.

As a concrete example, consider Figure 4.3 and suppose the candidate vertex is A, with neighbor B under consideration. Here, $V_A^{\text{nbr}} = \{\text{blue}, \text{green}\}$, while $V_B^{\text{nbr}} = \{\text{blue}\}$. Consequently, $V_{AB}^{A} = \{\text{green}\}$, since green is the only partition connected to A but not to B, whereas $V_{AB}^{\text{nbr}} = \{\text{blue}\}$ as both vertices have at least one edge to the blue partition. Keeping A as the candidate vertex, consider now neighbor C with $V_C^{\text{nbr}} = \{\text{red}, \text{green}\}$. A and C have access to the same partitions, meaning that $V_{AC}^{A} = \emptyset$ and $V_{AC}^{\text{nbr}} = \{\text{blue}, \text{green}\}$. This example highlights the need for adding $P[v]$ to the sets. The sets would have been $V_{AC}^{A} = \{\text{blue}\}$, $V_{AC}^{\text{nbr}} = \{\text{green}\}$ if we had not accounted for $P[C]$. However, by simply viewing Figure 4.3 we can see that the blue partition is shared and not unique to A.

Let $C_i[n]$ denote the number of edges connecting i to partition n. In Figure 4.3, for example, $C_A[\text{blue}] = 1$, as A has only one edge connected to the blue partition, while $C_D[\text{blue}] = 2$, as D has two edges connected to the blue partition. Let G_i be a vector with a number of elements equal to the number of partitions, initialized at 0, where $G_i[n]$ represents the total gain obtained by moving vertex i to partition n. This structure is used by METIS to store for each candidate node, the resulting change of moving to a partition. For a candidate vertex i being considered moving, the gain contributions are evaluated for each neighbor $v \in V_i^{\text{adj}}$ as follows:

Case 1. If $P[v] = P[i]$, then moving i to any partition in $k \in V_{iv}^{i}$ creates a shared inter-partition connection for v, yielding

$$G_i[k] -= \text{vsize}(i) \tag{4.3}$$

Case 2. If $P[v] \neq P[i]$ and $C_v[P[i]] = 1$, then moving i to any partition $k \in V_{iv}^{\text{nbr}}$ eliminates an inter-partition connection for v, yielding

$$G_i[k] += \text{vsize}(i) \tag{4.4}$$

Case 3. If $P[v] \neq P[i]$ and $C_v[P[i]] > 1$, then moving i to any partition $k \in V_{iv}^{i}$ introduces a new inter-partition connection for v, resulting in a gain contribution of

$$G_i[k] -= \text{vsize}(i) \tag{4.5}$$

Case 4. If $C_i[P[i]] = 0$, i.e., i has no internal degree, then moving i to any partition $k \in V_i^{nbr}$ yields an additional gain contribution of

$$G_i[k] += \text{vsize}(i) \tag{4.6}$$

For every adjacent boundary vertex v that it visits, METIS will check each of the cases above for every partition $k \in V_i^{\text{nbr}}$. The algorithm below displays the flow of how METIS evaluates the gain for a boundary vertex:

Algorithm 1 METIS gain calculation

```
partition_i ← P[i]
Nbr_i ← i.nbr_partitions                                  ▷ Adjacent partitions of i
Adj ← i.adj_vertices                                      ▷ Adjacent vertices of i
C_i ← i.connections        ▷ Vector where each element represents the edges i has to a
partition
vsize ← i.size
for all v ∈ Adj do
    partition_v ← P[v]
    Nbr_v ← v.nbr_partitions
    Nbr_v.add(P[v])
    C_v ← v.connections
    for n = 0; n < nparts do
        if n in Nbr_i AND n in Nbr_v then
            V_nbr.add(n)                  ▷ Shared partitions between i and v (V_iv^nbr)
        else if n in Nbr_i AND n NOT IN Nbr_v then
            V_i.add(n)                        ▷ Partitions only i is connected to (V_iv^i)
        end if
    end for
    if partition_i = partition_v then
        for all n ∈ V_i do
            G[n] −= vsize
        end for
    else if partition_i != partition_v AND C_v[partition_i] = 1 then
        for all n ∈ V_nbr do
            G[n] += vsize
        end for
    else if partition_i != partition_v AND C_v[partition_i] > 1 then
        for all n ∈ V_i do
            G[n] −= vsize
        end for
    end if
    if C_i[partition_i] = 1 then
        for all n ∈ Nbr_i do
            G[n] += vsize
        end for
    end if
end for
```

4.3 Partition specific gain calculation

To analyze communication at the partition level, we needed to expand the strategy used by METIS as shown in section 4.2. By only having the global gain for a move, information regarding how this gain affected each partition was missing.

In order to calculate the volume for a given partition n, consider a subset of boundary vertices where each vertex is part of the same partition. The communication volume associated with partition n is then given by

$$\text{partition_volume}_n = \sum_{v \in V_b | P[v]=n} \left| V_v^{\text{nbr}} \right| \text{vsize(v)} \tag{4.7}$$

We introduce a matrix $SG_i \in \mathbb{R}^{m \times m}$, where each row corresponds to a source partition and each column to a destination partition for vertex i. The entry $SG_i[n][k]$ represents the gain contribution to partition n when i is moved to partition k. In a graph with m partitions, a vertex i to partition n the total gain satisfies

$$\sum_{t=0}^{m-1} SG_i[t][n] = G_i[n] \tag{4.8}$$

i.e., the sum of each column in SG_i equals the corresponding entry in SG_i. The partition-specific gain contributions are then given as follows:

Case 1. If $P[v] = P[i]$, then moving i to any partition $k \in V_{iv}^i$ removes a shared inter-partition connection. This affects the source partition:

$$SG_i[P[i]][k] -= \text{vsize}(i) \tag{4.9}$$

Case 2. If $P[v] \neq P[i]$ and $C_v[P[i]] = 1$, then moving i to any partition $k \in V_{iv}^{\text{nbr}}$ eliminates an inter-partition connection. This contributes to partition $P[v]$:

$$SG_i[P[v]][k] += \text{vsize}(i) \tag{4.10}$$

Case 3. If $P[v] \neq P[i]$ and $C_v[P[i]] > 1$, then moving i to any partition $k \in V_{iv}^i$, introduces a new inter-partition connection. This contributes negatively to partition $P[v]$:

$$SG_i[P[v]][k] -= \text{vsize}(i) \tag{4.11}$$

Case 4. If $C_i[P[i]] = 0$, then moving i to any partition $k \in V_i^{nbr}$ yields an additional gain contribution for the destination partition:

$$SG_i[k][k] += \text{vsize}(i) \tag{4.12}$$

It is also necessary to account for how both the source and destination partitions are affected by a vertex move. Although such a move redistributes communication volume between partitions, it does not alter the total communication volume of the graph. As discussed previously, NVOL requires precise tracking of how each move impacts individual partitions. When vertex i is moved to partition k, this induces an additional gain contribution given by

$$SG_i[P[i]][k] += \left| V_i^{\text{nbr}} \right| \cdot \text{vsize}(i), \tag{4.13}$$

$$SG_i[k][k] -= \left| V_i^{\text{nbr}} \right| \cdot \text{vsize}(i). \tag{4.14}$$

This is referred to as a global term operation, as it reflects the overall redistribution of communication volume between the source and destination partitions of vertex i, rather than interactions with individual neighbor vertices. Consequently, it is computed once per vertex move, in contrast to the neighbor specific gain contributions described earlier, which are evaluated separately for each adjacent vertex.

Apart from this distinction, both strategies follow the same case structure; the principal difference lies in how they store the gain information. Rather than recording only the magnitude of the change, NVOL also tracks which partition is affected.

4.4 Examples

To build intuition for the definitions of communication volume and gain, we present a sequence of illustrative examples. Each example highlights a particular configuration and demonstrates how gain contributions arise under different vertex moves.

4.4.1 Two Partition example

Consider the graph in Figure 4.4, consisting of five vertices with unit weights. The graph is partitioned into two subsets: the red partition containing vertices $\{A, B, C\}$, and the blue partition containing vertices $\{D, E\}$. Using Equation 4.1, the total communication volume is 4.

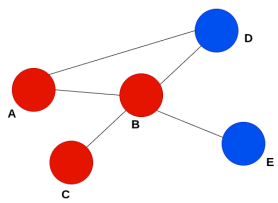

Figure 4.4: Graph with a volume of 4, each partition has a volume of 2.

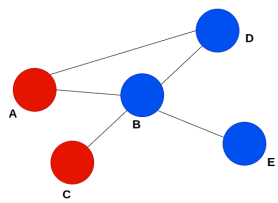

Figure 4.5: Graph with a volume of 4, each partition has a volume of 2 after the move.

Vertex A has one edge to the blue partition and contributes 1 to the communication volume. Vertex B has two edges incident to the blue partition, but contributes only 1 under the given definition. Similarly, vertices D and E each contribute 1. In contrast, vertex C contributes nothing, as all its incident edges are internal to its partition.

Now consider moving vertex B from the red partition to the blue partition, as shown in Figure 4.5. We calculate the gain using the METIS strategy (Section 4.2). Let B be the candidate vertex with neighbors $\{A, C, D, E\}$. Since there are two partitions, the gain vector $G_B \in \mathbb{R}^2$ has size two.

Vertex *A*. Vertices A and B are in the same partition and both already have connections to the blue partition. Hence, the set M is empty and no gain contribution arises.

Vertex *C*. Vertex C lies in the red partition and has no edges to the blue partition. After moving B, C gains an inter-partition edge, increasing the communication volume. As shown in Case , this yields a negative gain:

$$G_B[\text{blue}] -= 1.$$

Vertex *D*. Vertex D is in the blue partition meaning that a move to the blue partition would decrease its volume only if one edge were incident on the red partition. Since D has two, Case does not apply, moving B does not change its contribution, hence no case applies.

Vertex *E*. Vertex E satisfies Case : it has exactly one connection to the red partition. After the move, this connection disappears, reducing the communication volume and yielding a positive gain:

$$G_B[\text{blue}] \mathrel{+}= 1.$$

Case does not apply, as B has multiple connections within its original partition. Summing all contributions yields total gain 0, so the communication volume remains unchanged.

Partition-specific formulation. Using the formulation from Section 4.3, we obtain the same result. We use matrix $SG_B \in \mathbb{R}^{2\times 2}$ to store each volume change. As shown above only vertices C and E contribute:

$$SG_B[\text{red}][\text{blue}] \mathrel{-}= 1, \quad SG_B[\text{blue}][\text{blue}] \mathrel{+}= 1.$$

Additionally, accounting for the direct effect of moving B via Equations 4.13 and 4.14:

$$SG_B[\text{red}][\text{blue}] \mathrel{+}= 1, \quad SG_B[\text{blue}][\text{blue}] \mathrel{-}= 1.$$

The net effect is zero, confirming that neither total nor per-partition volume changes.

4.4.2 Three partition example

We now extend the example to three partitions. Consider the graph in Figure 4.6, where all vertices have unit weight. The partitions are: red $\{A, B\}$, blue $\{C, D\}$, and green $\{E\}$. The total communication volume is 8, with partition volumes 3, 3, and 2, respectively.

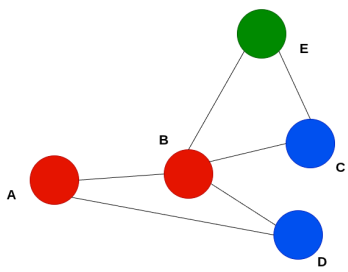

Figure 4.6: Graph with a total communication volume of 8.

Let B be the candidate vertex with neighbors $\{A, C, D, E\}$. Unlike the previous example, B may move to either the blue or green partition. Thus, $G_B \in \mathbb{R}^3$ and $SG_B \in \mathbb{R}^{3\times 3}$.

Vertex *A*. Vertex A is in the same partition as B. Since A is connected to the blue partition but not the green partition, Case applies only for a move to green, introducing a new inter-partition edge:

$$G_B[\text{green}] \mathrel{-}= 1.$$

Vertex *C*. Vertex C is outside the red partition and has exactly one edge to it. By Case , removing this edge reduces the communication volume. Since C is connected to both the blue and green partitions:

$$G_B[\text{blue}] \mathrel{+}= 1, \quad G_B[\text{green}] \mathrel{+}= 1.$$

Vertex *D*. Vertex D has two edges to the red partition. By Case , a gain change occurs only when the destination partition is not already adjacent to D. This holds for the green partition:

$$G_B[\text{green}] \mathrel{-}= 1.$$

Vertex *E*. Vertex E is symmetrical to C, yielding:

$$G_B[\text{blue}] \mathrel{+}= 1, \quad G_B[\text{green}] \mathrel{+}= 1.$$

Summing contributions gives:

$$G_B = [0, 2, 0],$$

indicating that moving B to the blue partition reduces the total communication volume by 2, while moving it to the green partition leaves it unchanged. Here

Partition-specific formulation. The vector G_B alone does not capture how the volume is redistributed among individual partitions. Using Section 4.3, we compute SG_B as follows.

Vertex *A*. Vertex A follows Case for a move to the green partition, yielding:

$$SG_B[\text{red}][\text{green}] \mathrel{-}= 1$$

Vertex *C*. Vertex C follows Case for both moves, yielding:

$$SG_B[\text{blue}][\text{blue}] \mathrel{+}= 1$$
$$SG_B[\text{blue}][\text{green}] \mathrel{+}= 1$$

Vertex *D*. Vertex D falls under Case for the green partition, yielding:

$$SG_B[\text{blue}][\text{green}] \mathrel{-}= 1,$$

Vertex *E*. Vertex E is symmetrical to C, yielding:

$$SG_B[\text{green}][\text{blue}] \mathrel{+}= 1$$
$$SG_B[\text{green}][\text{green}] \mathrel{+}= 1$$

Additionally, accounting for the direct effect of moving B via Equations 4.13 and 4.14:

$$SG_B[\text{red}][\text{blue}] \mathrel{+}= 2, \quad SG_B[\text{red}][\text{green}] \mathrel{+}= 2,$$

$$SG_B[\text{blue}][\text{blue}] \mathrel{-}= 2, \quad SG_B[\text{green}][\text{green}] \mathrel{-}= 2.$$

The resulting matrix SG_B is:

$$\begin{array}{c} \\ \text{red} \\ \text{blue} \\ \text{green} \end{array} \begin{array}{c} \begin{array}{ccc} \text{red} & \text{blue} & \text{green} \end{array} \\ \begin{pmatrix} 0 & 2 & 1 \\ 0 & -1 & 0 \\ 0 & 1 & -1 \end{pmatrix} \end{array}$$

Each column corresponds to a destination partition, and each row describes how a partition's volume changes. One can verify that the column sums recover G_B: the sums are $0+0+0=0$, $2+(-1)+1=2$, and $1+0+(-1)=0$, consistent with $G_B=[0,2,0]$. This formulation provides finer insight: although moving B to the green partition leaves the total volume unchanged, it redistributes volume across partitions. In particular, the blue partition experiences an increase in volume at the partition level.

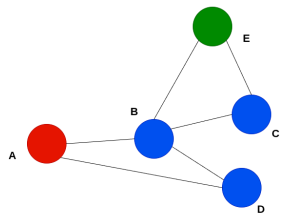

Figure 4.7: Graph has a volume of 6 after B is moved to the blue partition

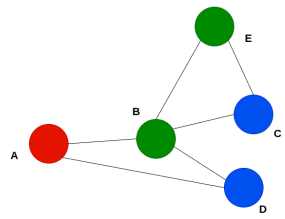

Figure 4.8: Graph has a volume of 8 after B is moved to the green partition

4.4.3 Case 4 example

None of the preceding examples illustrate a scenario in which Case 4 is relevant. To clarify how such a case may arise and how it affects the communication volume, consider Figure 4.9, where all vertices carry unit weight. The partition assignment is as follows: red $\{A,B,C,D\}$, blue $\{D,E\}$, and green $\{F\}$. The total communication volume is 8, with individual partition volumes of 3, 3, and 2, respectively. As in the previous example, let $G_C \in \mathbb{R}^3$ and $SG_C \in \mathbb{R}^{3\times 3}$.

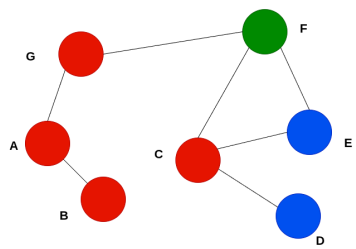

Figure 4.9: Graph with a total communication volume of 8.

The candidate vertex is C, which has no internal edges within the red partition. Despite the graph containing additional vertices, only the direct neighbors $\{D,E,F\}$ of C need to be visited in order to compute the gain. This illustrates how, even in large graphs, gain computation remains local to the immediate neighborhood of the moving vertex. For conciseness, both computation strategies are consolidated in this example, as they have been thoroughly demonstrated in earlier sections.

Vertex *D*. Vertex D satisfies Case 2 for a move to the blue partition, yielding:

$$\begin{aligned} G_C[\text{blue}] &+= 1 \\ SG_C[\text{blue}][\text{green}] &+= 1 \end{aligned}$$

Vertex *E*. Vertex E satisfies Case 2 for all possible moves of C, yielding:

$$\begin{aligned} G_C[\text{blue}] &+= 1 \\ G_C[\text{green}] &+= 1 \\ SG_C[\text{blue}][\text{blue}] &+= 1 \\ SG_C[\text{blue}][\text{green}] &+= 1 \end{aligned}$$

Vertex *F*. Vertex F has two edges connecting it to the red partition; however, the set M is empty, meaning Case 3 does not apply and vertex F contributes no change to the gain

Case 4. Since vertex C has no internal edges, any move will reduce the volume of its source partition. Under normal circumstances, when the candidate vertex has at least one internal edge, moving it leaves the receiving partition's incoming volume unchanged through the candidate. In this case, however, it is necessary to increment the gain for every possible destination partition, yielding:

$$\begin{aligned} G_C[\text{blue}] &+= 1 \\ G_C[\text{green}] &+= 1 \\ SG_C[\text{blue}][\text{blue}] &+= 1 \\ SG_C[\text{green}][\text{green}] &+= 1 \end{aligned}$$

The resulting vector $G_C = [0\,3\,2]$. Accounting for the direct effect of relocating C via Equations 4.13 and 4.14:

$$SG_C[\text{red}][\text{blue}] += 2, \quad SG_C[\text{red}][\text{green}] += 2,$$

$$SG_C[\text{blue}][\text{blue}] -= 2, \quad SG_C[\text{green}][\text{green}] -= 2.$$

The resulting matrix SG_C is:

$$\begin{array}{c} \\ \text{red} \\ \text{blue} \\ \text{green} \end{array} \begin{array}{c} \begin{array}{ccc} \text{red} & \text{blue} & \text{green} \end{array} \\ \begin{pmatrix} 0 & 2 & 2 \\ 0 & 1 & 1 \\ 0 & 1 & -1 \end{pmatrix} \end{array}$$

Figures 4.10 and 4.11 show the resulting graphs after moving C to the blue and green partitions respectively. The reader can verify that the partition volumes in each case are consistent with the gains predicted by SG_C and G_C. Without accounting for Case 4, the gain calculation would be incomplete, potentially leading to incorrect volume estimates when a vertex has no internal edges.

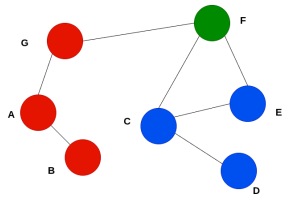

Figure 4.10: Graph has a volume of 5 after the move.

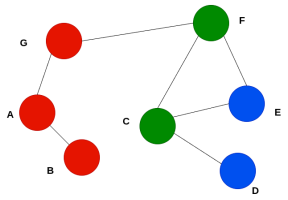

Figure 4.11: Graph has a volume of 6 after the move.

4.5 Implementation

Having verified the formulation across several configurations, we now describe how it is integrated into the program. The implementation closely mirrors the case structure presented above

	NVOL02	NVOL02 4%	NVOL03	NVOL03 4%
heart01	-0.2	-10.2	-10.1	-10.1
heart02	+8.2	-0.2	+6.4	0.0
heart03	-2.7	-2.1	+7.7	-1.5
heart04	+11.4	+3.4	+3.3	+8.7
heart05	+0.8	+29.2	+4.2	+34.4
heart06	-1.1	-5.8		-4.6
heart07	-3.6	-14.3		-7.5
channel-500x100x100-b050	-10.8	-23.6	-16.5	-11.1
fe_tooth	-6.7	+1.3	-5.4	-5.1
hugetrace-00020	-2.5	+18.9	-2.2	
hugetric-00020	-1.9	+8.9	-1.7	+8.8
hugetric-00000	-5.7	+1.4	-5.0	+0.9
citationCiteseer	+3.5	-23.6	+8.5	-17.8
italy_osm	0.0	+27.1	0.0	+27.2
europe_osm	0.0	+61.0	0.0	
roadNet-CA	-0.4	-0.2	0.0	12.6
kmer_U1a	-2.4	-7.4	+5.7	+0.6
kmer_V2a	0.0	+31.9	+1.5	+39
pa2010	0.0	+0.2		+1.0
coPapersDBLP	+4.0	-0.4		+10.2

Table 4.1: $k = 32$

Chapter 5

Results

Chapter 6

Conclusions and Future Work

Chapter 7

Results

Chapter 8

Future Work

Bibliography

[1] Stephen T. Barnard and Horst D. Simon. 'Fast multilevel implementation of recursive spectral bisection for partitioning unstructured problems'. In: *Concurrency: Practice and Experience* 6.2 (1994), pp. 101–117. DOI: https://doi-org.ezproxy.uio.no/10.1002/cpe.4330060203.

[2] Erik G. Boman, Karen D. Devine and Sivasankaran Rajamanickam. 'Scalable matrix computations on large scale-free graphs using 2D graph partitioning'. In: *SC '13: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*. 2013, pp. 1–12. DOI: 10.1145/2503210.2503293.

[3] Aydın Buluç, Kamesh Madduri, Henning Meyerhenke, Peter Sanders, David A. Bader and Dorothea Wagner. 'Graph partitioning for scalable distributed graph computations'. In: *Graph Partitioning and Graph Clustering*. Vol. 588. Contemporary Mathematics. American Mathematical Society, 2013, pp. 83–101. DOI: 10.1090/conm/588/11709.

[4] Aydın Buluç, Henning Meyerhenke, Ilya Safro, Peter Sanders and Christian Schulz. 'Recent Advances in Graph Partitioning'. In: *Algorithm Engineering: Selected Results and Surveys*. Springer International Publishing, 2016, pp. 117–158. DOI: 10.1007/978-3-319-49487-6_4. URL: https://doi.org/10.1007/978-3-319-49487-6_4.

[5] U.V. Catalyurek and C. Aykanat. 'Hypergraph-partitioning-based decomposition for parallel sparse-matrix vector multiplication'. In: *IEEE Transactions on Parallel and Distributed Systems* 10.7 (1999), pp. 673–693. DOI: 10.1109/71.780863.

[6] Ümi˙t V. Çatalyürek, Cevdet Aykanat and Bora Uçar. 'On Two-Dimensional Sparse Matrix Partitioning: Models, Methods, and a Recipe'. In: *SIAM Journal on Scientific Computing* 32.2 (2010), pp. 656–683. DOI: 10.1137/080737770.

[7] Ümit Çatalyürek et al. 'More Recent Advances in (Hyper)Graph Partitioning'. In: 55.12 (2023). DOI: 10.1145/3571808.

[8] Ümit V. Çatalyürek, Mehmet Deveci, Kamer Kaya, Bora Uçar, Henning Meyerhenke, Peter Sanders, David A. Bader and Dorothea Wagner. 'UMPa: A multi-objective, multi-level partitioner for communication minimization'. In: *Graph Partitioning and Graph Clustering*. Vol. 588. Contemporary Mathematics. American Mathematical Society, 2013, pp. 53–65. DOI: 10.1090/conm/588/11704.

[9] Jingsheng Jason. Cong and Joseph R. Shinnerl. *Multilevel Optimization in VLSICAD*. 1st ed. 2003. Combinatorial Optimization. Springer US, 2003. DOI: 10.1007/978-1-4757-3748-6.

[10] Ralf Diekmann, Robert Preis, Frank Schlimbach and Chris Walshaw. 'Shape-optimized mesh partitioning and load balancing for parallel adaptive FEM'. In: *Parallel Computing* 26.12 (2000). Graph Partitioning and Parallel Computing, pp. 1555–1581. ISSN: 0167-8191. DOI: https://doi.org/10.1016/S0167-8191(00)00043-0.

[11] Executive Office of the President. *Executive Order 13702—Creating a National Strategic Computing Initiative.* Federal Register, Vol. 80, No. 149, pp. 46177–46180. July 2015. URL: https://www.federalregister.gov/documents/2015/08/03/2015-19183/creating-a-national-strategic-computing-initiative (visited on 13/04/2026).

[12] Miroslav Fiedler. 'A property of eigenvectors of nonnegative symmetric matrices and its application to graph theory'. In: *Czechoslovak mathematical journal* 24.4 (1975), pp. 619–633. DOI: 10.21136/cmj.1975.101357.

[13] Santo Fortunato. 'Community detection in graphs'. In: *Physics Reports* 486.3 (2010), pp. 75–174. DOI: https://doi.org/10.1016/j.physrep.2009.11.002.

[14] M.R. Garey, D.S. Johnson and L. Stockmeyer. 'Some simplified NP-complete graph problems'. In: *Theoretical Computer Science* 1.3 (1976), pp. 237–267. DOI: https://doi.org/10.1016/0304-3975(76)90059-1.

[15] Michael S. Gilbert, Kamesh Madduri, Erik G. Boman and Siva Rajamanickam. 'Jet: Multilevel Graph Partitioning on Graphics Processing Units'. In: *SIAM Journal on Scientific Computing* 46.5 (2024), B700–B724. DOI: 10.1137/23M1559129.

[16] Bruce Hendrickson. 'Graph partitioning and parallel solvers: Has the emperor no clothes?' In: *Solving Irregularly Structured Problems in Parallel.* Springer Berlin Heidelberg, 1998, pp. 218–225. DOI: 10.1007/BFb0018541.

[17] Bruce Hendrickson and Tamara G Kolda. 'Graph partitioning models for parallel computing'. In: *Parallel Computing* 26.12 (2000). Graph Partitioning and Parallel Computing, pp. 1519–1534. DOI: https://doi.org/10.1016/S0167-8191(00)00048-X.

[18] Bruce Hendrickson and Robert Leland. 'A multilevel algorithm for partitioning graphs'. In: *Proceedings of the 1995 ACM/IEEE Conference on Supercomputing.* Supercomputing '95. ACM, 1995, pp. 28–42. DOI: 10.1145/224170.224228.

[19] John L. Hennessy and David A. Patterson. 'A new golden age for computer architecture'. In: *Commununications of the ACM* 62.2 (2019), pp. 48–60. DOI: 10.1145/3282307.

[20] Costas Iliopoulos, Jason T. L Wang, Mourad Elloumi and Albert Y Zomaya. 'Wiley Series on Bioinformatics: Computational Techniques and Engineering'. In: *Pattern Recognition in Computational Molecular Biology.* John Wiley & Sons, Incorporated, 2015. DOI: 10.1002/9781119078845.scard.

[21] Andrew B. Kahng, Jens. Lienig, Igor L. Markov and Jin. Hu. *VLSI Physical Design: From Graph Partitioning to Timing Closure.* 1st ed. 2011. Springer Netherlands, 2011. DOI: 10.1007/978-90-481-9591-6.

[22] G. Karypis and V. Kumar. 'Analysis of Multilevel Graph Partitioning'. In: *Supercomputing '95:Proceedings of the 1995 ACM/IEEE Conference on Supercomputing.* 1995, pp. 29–29. DOI: 10.1145/224170.224229.

[23] George Karypis and Vipin Kumar. 'A Fast and High Quality Multilevel Scheme for Partitioning Irregular Graphs'. In: *SIAM Journal on Scientific Computing* 20.1 (1998), pp. 359–392. DOI: 10.1137/S1064827595287997.

[24] George Karypis and Vipin Kumar. 'Multilevelk-way Partitioning Scheme for Irregular Graphs'. In: *Journal of Parallel and Distributed Computing* 48.1 (1998), pp. 96–129. DOI: https://doi.org/10.1006/jpdc.1997.1404.

[25] George Karypis and Vipin Kumar. 'Parallel Multilevel series k-Way Partitioning Scheme for Irregular Graphs'. In: *SIAM Review* 41.2 (1999), pp. 278–300. DOI: 10.1137/S0036144598334138.

[26] George Karypis and Vipin Kumar. 'Parallel Multilevel series k-Way Partitioning Scheme for Irregular Graphs'. In: *SIAM Review* 41.2 (1999), pp. 278–300. DOI: 10.1137/S0036144598334138.

[27] Johannes Langguth, Sebastian Schlag and Christian Schulz. 'Load-Balanced Bottleneck Objectives in Process Mapping'. In: (2020). DOI: 10.48550/arXiv.2001.09645.

[28] Hao Li, G.W. Rosenwald, J. Jung and Chen-ching Liu. 'Strategic Power Infrastructure Defense'. In: *Proceedings of the IEEE* 93.5 (2005), pp. 918–933. DOI: 10.1109/JPROC.2005.847260.

[29] Juan Li and Chen-Ching Liu. 'Power system reconfiguration based on multilevel graph partitioning'. In: *2009 IEEE Bucharest Power Tech.* IEEE, 2009, pp. 1–5. DOI: 10.1109/PTC.2009.5281971.

[30] Irene Moulitsas, George Karypis, Anu G Bourgeois and Si Quing Zheng. 'Architecture Aware Partitioning Algorithms'. In: *Algorithms and Architectures for Parallel Processing.* Vol. 5022. Lecture Notes in Computer Science. Springer Berlin / Heidelberg, 2008, pp. 42–53. DOI: 10.1007/978-3-540-69501-1_6.

[31] M. E. J. Newman. 'Community detection and graph partitioning'. In: *Europhysics Letters* 103.2 (2013). DOI: 10.1209/0295-5075/103/28003.

[32] M. E. J. Newman. *Networks: An Introduction.* Oxford University Press, 2010. DOI: 10.1093/acprof:oso/9780199206650.001.0001.

[33] Joel Nishimura and Johan Ugander. 'Restreaming graph partitioning: simple versatile algorithms for advanced balancing'. In: *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining.* KDD '13. ACM, 2013, pp. 1106–1114. DOI: 10.1145/2487575.2487696.

[34] Fareed Qararyah, Mohamed Wahib, Doğa Dikbayır, Mehmet Esat Belviranli and Didem Unat. 'A computational-graph partitioning method for training memory-constrained DNNs'. In: *Parallel Computing* 104-105 (2021), pp. 1–12. DOI: https://doi.org/10.1016/j.parco.2021.102792.

[35] Semih Salihoglu, Jennifer Widom, Magdalena Balazinska, Alex Szalay, Ahmet Sacan, Tamas Budavari and Alexandra Meliou. 'GPS: a graph processing system'. In: *Proceedings of the 25th International Conference on Scientific and Statistical Database Management.* ACM, 2013, pp. 1–12. DOI: 10.1145/2484838.2484843.

[36] Xianfeng Song, Yi Zou and Zheng Shi. 'CaPGNN: Optimizing parallel graph neural network training with joint caching and resource-aware graph partitioning'. In: *Neurocomputing (Amsterdam)* 675 (2026), pp. 1–15. DOI: 10.1016/j.neucom.2026.132978.

[37] Isabelle Stanton and Gabriel Kliot. 'Streaming graph partitioning for large distributed graphs'. In: *Proceedings of the 18th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining.* KDD '12. ACM, 2012, pp. 1222–1230. DOI: 10.1145/2339530.2339722.

[38] Herb Sutter and James Larus. 'Software and the concurrency revolution'. In: *ACM Queue* 3.7 (2005), pp. 54–62. DOI: 10.1145/1095408.1095421.

[39] Charalampos Tsourakakis, Christos Gkantsidis, Bozidar Radunovic and Milan Vojnovic. 'FENNEL: streaming graph partitioning for massive scale graphs'. In: *Proceedings of the 7th ACM International Conference on Web Search and Data Mining*. WSDM '14. ACM, 2014, pp. 333–342. DOI: 10.1145/2556195.2556213.

[40] C. Walshaw, M. Cross and M.G. Everett. 'Parallel Dynamic Graph Partitioning for Adaptive Unstructured Meshes'. In: *Journal of Parallel and Distributed Computing* 47.2 (1997), pp. 102–108. DOI: https://doi.org/10.1006/jpdc.1997.1407.